Status	Finished
Started	Friday, 30 October 2026, 3:42 PM
Completed	Friday, 30 October 2026, 3:46 PM
Duration	4 mins 10 secs
Grade	**15.00** out of 15.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

The readlines() method returns _____

a. str

b. a list of lines ✓

c. a list of single characters

d. a list of integers

The correct answer is:
a list of lines

Question **2**

Correct

Mark 1.00 out of 1.00

What does the buffering parameter in the open() function control?

a. The buffer size for the file operations ✓

b. The file's read/write position

c. The encoding of the file

d. The access mode of the file

The correct answer is: The buffer size for the file operations

Question **3**

Correct

Mark 1.00 out of 1.00

If the file is opened in write mode and already exists, it truncates the existing content and places the filehandle at the beginning of the file.

Select one:

- True ✓
- False

The correct answer is 'True'.

Question **4**

Correct

Mark 1.00 out of 1.00

Which of the following modes opens a file for both reading and writing?

- a. w
- b. r
- c. a
- d. r+ ✓

The correct answer is: r+

Question **5**

Correct

Mark 1.00 out of 1.00

To open a file a.txt for reading, we use _____

- a. infile = open(file = “a.txt”, “r”)
- b. infile = open(file, “a.txt”, “r”)
- c. infile = open(“a.txt”, “r”) ✓
- d. infile = open(file = “a.txt”, “r”)

The correct answer is:
infile = open(“a.txt”, “r”)

Question **6**

Correct

Mark 1.00 out of 1.00

What is the output of the following code?

```
f = open("test.txt", "w")
f.write("Hello, world!")
f.close()
f = open("test.txt", "r")
print(f.read())
f.close()
```

- a. Hello, world! ✓
- b. An error is raised
- c. world
- d. Hello

The correct answer is: Hello, world!

Question **7**

Correct

Mark 1.00 out of 1.00

Which method is used to write a list of strings to a file?

- a. write_all()
- b. writelines() ✓
- c. write()
- d. write_list()

The correct answer is: writelines()

Question **8**

Correct

Mark 1.00 out of 1.00

What happens if you try to open a file in write mode ('w') and the file already exists?

a. It appends data to the file
b. It raises an error
c. It does nothing
d. It clears the old data and starts fresh ✓

The correct answer is: It clears the old data and starts fresh

Question **9**

Correct

Mark 1.00 out of 1.00

What does the flush() method do in file handling?

a. Deletes the file content
b. Moves the file cursor to the beginning
c. Flushes the internal buffer to the file ✓
d. Closes the file

The correct answer is: Flushes the internal buffer to the file

Question **10**

Correct

Mark 1.00 out of 1.00

Which method is used to sets the position of a file pointer

a. tell()
b. ftell()
c. fseek()
d. seek() ✓

The correct answer is: seek()

Question **11**

Correct

Mark 1.00 out of 1.00

Which method is used to write data to a file?

- a. read()
- b. insert()
- c. write() ✓
- d. append()

The correct answer is: write()

Question **12**

Correct

Mark 1.00 out of 1.00

Which method can be used to read all lines of a file into a list?

- a. readlines() ✓
- b. readline()
- c. readlist()
- d. read()

The correct answer is: readlines()

Question **13**

Correct

Mark 1.00 out of 1.00

Which method is used to get the current file position?

- a. position()
- b. current()
- c. tell() ✓
- d. locate()

The correct answer is: tell()

Question **14**

Correct

Mark 1.00 out of 1.00

What is the difference between r+ and w+ modes in file handling?

a. r+ is for binary read and write, w+ is for text read and write
b. r+ is for read and append, w+ is for read and write
c. r+ is for read and write, w+ is for append only
d. r+ is for read and write without truncating, w+ is for read and write with truncating ✓

The correct answer is: r+ is for read and write without truncating, w+ is for read and write with truncating

Question **15**

Correct

Mark 1.00 out of 1.00

What is the use of tell() method in python?

a. tells you the current position within the file ✓
b. tells you the file is opened or not
c. none of the mentioned
d. tells you the end position within the file

The correct answer is:
tells you the current position within the file

Status	Finished
Started	Friday, 30 October 2026, 4:09 PM
Completed	Friday, 30 October 2026, 4:17 PM
Duration	8 mins 11 secs
Grade	**10.00** out of 10.00 (**100**%)

Question **1**

Correct

Mark 1.00 out of 1.00

Write a Python program to count the frequency of each word in a given text file.

Description:

1. **Input:**
   - String as input.
2. **Output:**
   - A list of words with their corresponding frequency count to be write in a file "output.txt"

Example:

- **Input File Content:**

  apple orange apple banana apple orange

Output:

apple: 3
orange: 2
banana: 1

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`apple orange apple banana apple orange`	`apple: 3`
		`banana: 1`
		`orange: 2`

Answer: (penalty regime: 0 %)

```
import string
from collections import Counter

def count_word_frequency():
    # Read the input line
    text = input()

    # Clean words: convert to lowercase and strip punctuation
    words = []
    for word in text.split():
        cleaned_word = word.strip(string.punctuation).lower()
        if cleaned_word:
            words.append(cleaned_word)

    # Count word frequencies
    word_counts = Counter(words)

    # Write to output.txt in alphabetical order
    with open('output.txt', 'w') as file:
        for word in sorted(word_counts.keys()):
            file.write(f"{word}: {word_counts[word]}\n")

if __name__ == "__main__":
    count_word_frequency()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`apple orange apple banana apple orange`	`apple: 3`
			`banana: 1`
			`orange: 2`
				`apple: 3`
				`banana: 1`
				`orange: 2`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`Hello world! Hello everyone. Welcome to the world of programming.`	`everyone: 1`
			`hello: 2`
			`of: 1`
			`programming: 1`
			`the: 1`
			`to: 1`
			`welcome: 1`
			`world: 2`
				`everyone: 1`
				`hello: 2`
				`of: 1`
				`programming: 1`
				`the: 1`
				`to: 1`
				`welcome: 1`
				`world: 2`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`One fish two fish Red fish blue fish`	`blue: 1`
			`fish: 4`
			`one: 1`
			`red: 1`
			`two: 1`
				`blue: 1`
				`fish: 4`
				`one: 1`
				`red: 1`
				`two: 1`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **2**

Correct

Mark 1.00 out of 1.00

Develop a Python program to identify and print all palindrome words from a given text file.

Description:

1. **Input:**
   - A text file containing multiple words.
2. **Output:**
   - A list of palindrome words found in the file name as 'output.txt'.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt	madam
		arora
		malayalam

Answer: (penalty regime: 0 %)

```
import string

def find_palindromes():
    # Read the filename from input
    filename = input().strip()

    palindromes = []

    with open(filename, 'r') as file:
        text = file.read()
        words = text.split()

        for word in words:
            # Strip punctuation and convert to lowercase for palindrome check
            cleaned_word = word.strip(string.punctuation).lower()

            # Check if non-empty and reads the same forwards and backwards
            if cleaned_word and cleaned_word == cleaned_word[::-1]:
                palindromes.append(cleaned_word)

    # Write the identified palindromes to output.txt
    with open('output.txt', 'w') as file:
        for p in palindromes:
            file.write(f"{p}\n")

if __name__ == "__main__":
    find_palindromes()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt	madam
			arora
			malayalam
				madam
				arora
				malayalam
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **3**

Correct

Mark 1.00 out of 1.00

Develop a Python program to copy the contents of one file to another file.

Description:

1. **Input:**
   - Source file and destination file names.
2. **Output:**
   - The content of the source file copied to the destination file.

For example:

Test	Input	Result

`with open('output1.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	output1.txt
		This is the source file.
		It contains multiple lines of text.
		Here is another line.

Answer: (penalty regime: 0 %)

```
def copy_file():
    # Read the source and destination file names
    source_filename = input().strip()
    destination_filename = input().strip()

    # Read content from source file and write to destination file
    with open(source_filename, 'r') as source_file:
        content = source_file.read()

    with open(destination_filename, 'w') as dest_file:
        dest_file.write(content)

if __name__ == "__main__":
    copy_file()
```

	Test	Input	Expected	Got	

✓	`with open('output1.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		output1.txt
			This is the source file.
			It contains multiple lines of text.
			Here is another line.
				This is the source file.
				It contains multiple lines of text.
				Here is another line.
					✓

✓	`with open('output2.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		output2.txt
			Hello, world!
			Python programming is amazing.
			Let's copy this text to another file.
				Hello, world!
				Python programming is amazing.
				Let's copy this text to another file.
					✓

✓	`with open('output3.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input3.txt
		output3.txt
			Single line.	Single line.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **4**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a text file and count the total number of words in the file.

Description:

1. **Input:**
   - A text file containing several lines of text.
   - File name you should get as input.
2. **Output:**
   - The total number of words in the file.

For example:

Input	Result
input2.txt	Total words: 14
input3.txt	Total words: 15

Answer: (penalty regime: 0 %)

```
def count_words_in_file():
    # Read the filename from input
    filename = input().strip()

    # Read the contents of the file and count words
    with open(filename, 'r') as file:
        content = file.read()
        words = content.split()

    print(f"Total words: {len(words)}")

if __name__ == "__main__":
    count_words_in_file()
```

	Input	Expected	Got	
✓	input1.txt	Total words: 6	Total words: 6	✓
✓	input2.txt	Total words: 14	Total words: 14	✓
✓	input3.txt	Total words: 15	Total words: 15	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **5**

Correct

Mark 1.00 out of 1.00

Create a Python program to write to a specific line in a text file, replacing the existing content of that line.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to write to.
   - New content for the specified line.
2. **Output:**
   - The updated file with the specified line replaced by the new content in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Updated line two.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`2`
	`Updated line two.`
		`Line one.`
		`Updated line two.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
def replace_line_in_file():
    # Read the inputs: filename, line number, and new line content
    filename = input().strip()
    line_number = int(input().strip())
    new_content = input().rstrip("\r\n")

    # Read all lines from the specified input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Replace the target line (using 1-based indexing)
    if 1 <= line_number <= len(lines):
        lines[line_number - 1] = new_content + "\n"

    # Write the updated lines to output.txt
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == "__main__":
    replace_line_in_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input1.txt`
		`2`
		`Updated line two.`
			`Line one.`
			`Updated line two.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Updated line two.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		`input2.txt`
		`2`
		`Line B Updated.`
			`Line A.`
			`Line B Updated.`
			`Line C.`
				`Line A.`
				`Line B Updated.`
				`Line C.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **6**

Correct

Mark 1.00 out of 1.00

Write a Python program to reverse the contents of a specific line in a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to reverse.
2. **Output:**
   - The updated file with the specified line's contents reversed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

Output:

Line one.
Line two.
eerht eniL.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
		`Line one.`
		`Line two.`
		`eerht eniL.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
def reverse_line_in_file():
    # Read filename and line number
    filename = input().strip()
    line_number = int(input().strip())

    # Read lines from the file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Process target line if within range (1-based indexing)
    if 1 <= line_number <= len(lines):
        line = lines[line_number - 1].rstrip("\r\n")

        # Keep trailing punctuation intact at the end
        if line and line[-1] in ".!?,":
            punctuation = line[-1]
            text_to_reverse = line[:-1]
            reversed_line = text_to_reverse[::-1] + punctuation
        else:
            reversed_line = line[::-1]

        lines[line_number - 1] = reversed_line + "\n"

    # Write output to file
    with open('output.txt', 'w') as file:
```

```
        file.writelines(lines)

if __name__ == "__main__":
    reverse_line_in_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
			Line one.
			Line two.
			eerht eniL.
			Line four.
				Line one.
				Line two.
				eerht eniL.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		2
			Line A.
			B eniL.
			Line C.
				Line A.
				B eniL.
				Line C.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **7**

Correct

Mark 1.00 out of 1.00

Develop a Python program to read a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to read.
2. **Output:**
   - The content of the specified line.

input1.txt:

Line one.
Line two.
Line three.
Line four.

For example:

Input	Result
input1.txt 3	Line three.

Answer: (penalty regime: 0 %)

```
def read_specific_line():
    # Read filename and target 1-based line number
    filename = input().strip()
    line_number = int(input().strip())

    # Read all lines from the specified input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Print the specific line content if valid line number
    if 1 <= line_number <= len(lines):
        print(lines[line_number - 1].rstrip("\r\n"))

if __name__ == "__main__":
    read_specific_line()
```

	Input	Expected	Got	
✓	input1.txt 3	Line three.	Line three.	✓
✓	input2.txt 3	Line C.	Line C.	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **8**

Correct

Mark 1.00 out of 1.00

Create a Python program to find the longest word in a text file.

- **Input:**
  - A text file containing multiple lines of text.
- **Output:**
  - The longest word in the file.

For example:

Input	Result
input1.txt	Longest word: containing

Answer: (penalty regime: 0 %)

```
import string

def find_longest_word():
    # Read the filename from standard input
    filename = input().strip()

    longest_word = ""

    with open(filename, 'r') as file:
        text = file.read()
        words = text.split()

        for word in words:
            # Strip punctuation attached to words
            cleaned_word = word.strip(string.punctuation)
            if len(cleaned_word) > len(longest_word):
                longest_word = cleaned_word

    print(f"Longest word: {longest_word}")

if __name__ == "__main__":
    find_longest_word()
```

	Input	Expected	Got	
✓	input1.txt	Longest word: containing	Longest word: containing	✓
✓	input2.txt	Longest word: thousand	Longest word: thousand	✓
✓	input3.txt	Longest word: supercalifragilisticexpialidocious	Longest word: supercalifragilisticexpialidocious	✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **9**

Correct

Mark 1.00 out of 1.00

Write a Python program to append a new line at a specific position in a text file, shifting existing lines down.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to insert the new line at.
   - New content for the new line.
2. **Output:**
   - The updated file with the new line inserted at the specified position, shifting the existing lines down in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  3

  Inserted line..

Output:

Line one.
Line two.
Inserted line.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	`input1.txt`
	`3`
	`Inserted line.`
		`Line one.`
		`Line two.`
		`Inserted line.`
		`Line three.`
		`Line four.`

Answer: (penalty regime: 0 %)

```
def insert_line_in_file():
    # Read inputs
    filename = input().strip()
    line_number = int(input().strip())
    new_content = input().rstrip("\r\n")

    # Read existing lines from the input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Ensure the last line ends with a newline if file is non-empty
    if lines and not lines[-1].endswith("\n"):
        lines[-1] += "\n"

    # Insert new line at specified 1-based index
    insert_index = line_number - 1
    if insert_index >= len(lines):
        lines.append(new_content + "\n")
    else:
        lines.insert(insert_index, new_content + "\n")
```

```
21
22      # Write output to file
23      with open('output.txt', 'w') as file:
24          file.writelines(lines)
25
26  if __name__ == "__main__":
27      insert_line_in_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input1.txt
		3
		Inserted line.
			Line one.
			Line two.
			Inserted line.
			Line three.
			Line four.
				Line one.
				Line two.
				Inserted line.
				Line three.
				Line four.
					✓

✓	`with open('output.txt', 'r') as file:`
	`  text = file.read()`
	`  print(text)`
		input2.txt
		4
		Inserted line D.
			Line A.
			Line B.
			Line C.
			Inserted line D.
				Line A.
				Line B.
				Line C.
				Inserted line D.
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.

Question **10**

Correct

Mark 1.00 out of 1.00

Create a Python program to delete a specific line from a text file based on a given line number.

Description:

1. **Input:**
   - A text file with multiple lines.
   - A line number to delete.
2. **Output:**
   - The updated file with the specified line removed in file "output.txt".

Example:

- **Input File Content:**

  "Line one.
  Line two.
  Line three.
  Line four."
  2

  Updated line two.

Output:

Line one.
Line three.
Line four.

For example:

Test	Input	Result

`with open('output.txt', 'r') as file:`
`  text = file.read()`
`  print(text)`
	input1.txt
	2
		Line one.
		Line three.
		Line four.

Answer: (penalty regime: 0 %)

```
def delete_line_from_file():
    # Read the filename and 1-based line number to delete
    filename = input().strip()
    line_number = int(input().strip())

    # Read lines from the input file
    with open(filename, 'r') as file:
        lines = file.readlines()

    # Delete the specified line if within valid 1-based range
    if 1 <= line_number <= len(lines):
        del lines[line_number - 1]

    # Write remaining lines to output.txt
    with open('output.txt', 'w') as file:
        file.writelines(lines)

if __name__ == "__main__":
    delete_line_from_file()
```

	Test	Input	Expected	Got	

✓	`with open('output.txt', 'r') as file:`
	`    text = file.read()`
	`    print(text)`
		`input1.txt`
		`2`
			`Line one.`
			`Line three.`
			`Line four.`
				`Line one.`
				`Line three.`
				`Line four.`
					✓

✓	`with open('output.txt', 'r') as file:`
	`    text = file.read()`
	`    print(text)`
		`input2.txt`
		`3`
			`Line A.`
			`Line B.`
				`Line A.`
				`Line B.`
					✓

Passed all tests! ✓

Correct

Marks for this submission: 1.00/1.00.